

Chapter 4

Android User Interface

User Interface (UI) is everything that the user can see and interact with. Android provides a variety of pre-built UI components such as structured layout objects and UI controls that allow you to build the graphical user interface for your app. Android also provides other UI modules for special interfaces such as dialogs, notifications, and menus.

4.1 Introduction to Multiple Screen Size and Orientation Interfaces

4.1.1 Introduction to Multiple Screen Size

Android runs on a variety of devices that have different screen sizes and pixel densities, so your apps layout needs to be flexible. The system performs basic scaling and resizing to adapt your user interface to different screens, but there is more work you should do to ensure your UI gracefully adapts for each type of screen.

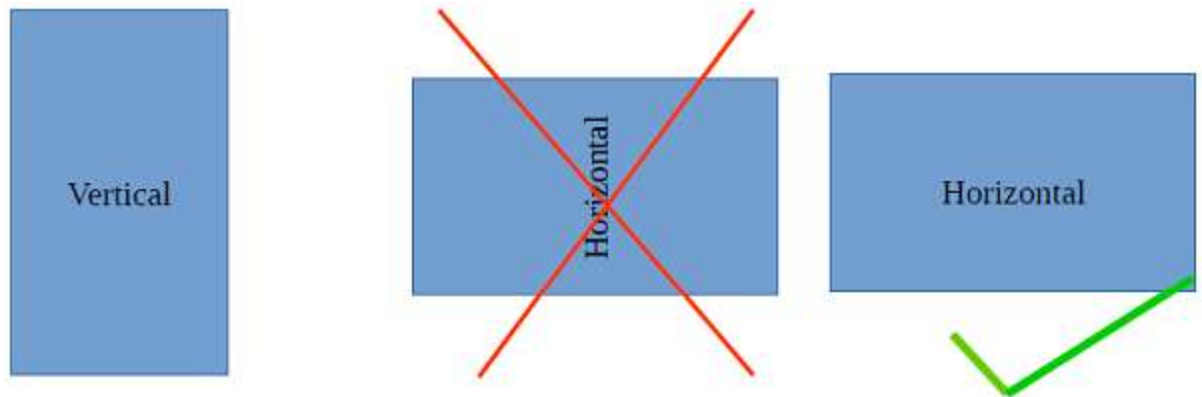
While making an application we should consider a) where the application will run and b) who is the user those are going to use that.

While making an application we should consider a) where the application will run and b) who is the user those are going to use that.

Normally, when we talk about Mobile phone, it has more height than width as it is designed vertically.

Therefore we may say we can design our application vertically but what if the user flip the screen and make it horizontal. Please see the screen on position 1 and position two.

4.1. INTRODUCTION TO MULTIPLE SCREEN SIZE AND ORIENTATION INTERFACES



Our screen should also be responsive to tackle horizontal position of the mobile. This is why android comes out with many layout and vertical/horizontal orientation to tackle those needs. **To design an UI you need to aware of the following terminologies**

1. Screen Size: - Actual physical size, measured as the screen's diagonal. For simplicity, Android groups all actual screen sizes into four generalized sizes: small, normal, large, and extra large.
 - small screens are at least 426dp x 320dp
 - normal screens are at least 470dp x 320dp
 - large screens are at least 640dp x 480dp
 - xlarge screens are at least 960dp x 720dp
2. Screen Density: - The quantity of pixels within a physical area of the screen; usually referred to as dpi (dots per inch). For example, a "low" density screen has fewer pixels within a given physical area, compared to a "normal" or "high" density screen. For simplicity, Android groups all actual screen densities into four generalized densities: low, medium, high, and extra high.
3. Orientation: - The orientation of the screen from the user's point of view. This is either landscape or portrait, meaning that the screen's aspect ratio is either wide or tall, respectively. Be aware that not only do different devices operate in different orientations by default, but the orientation can change at runtime when the user rotates the device.
4. Resolution: - The total number of physical pixels on a screen. When adding support for multiple screens, applications do not work directly with resolution; applications should be concerned only with screen size and density, as specified by the generalized size and density groups.

4.1. INTRODUCTION TO MULTIPLE SCREEN SIZE AND ORIENTATION INTERFACES

5. Density-independent pixel (dp): - A virtual pixel unit that you should use when defining UI layout, to express layout dimensions or position in a density-independent way.
The density-independent pixel is equivalent to one physical pixel on a 160 dpi screen, which is the baseline density assumed by the system for a "medium" density screen. At runtime, the system transparently handles any scaling of the dp units, as necessary, based on the actual density of the screen in use. The conversion of dp units to screen pixels is simple: $px = dp * (dpi / 160)$. For example, on a 240 dpi screen, 1 dp equals 1.5 physical pixels. You should always use dp units when defining your application's UI, to ensure proper display of your UI on screens with different densities.

4.1.2 How to Support Multiple Screens

The foundation of Android's support for multiple screens is its ability to manage the rendering of an application's layout and bitmap drawables in an appropriate way for the current screen configuration. The system handles most of the work to render your application properly on each screen configuration by scaling layouts to fit the screen size/density and scaling bitmap drawables for the screen density, as appropriate. To more gracefully handle different screen configurations, however, you should also:

- Explicitly declare in the manifest which screen sizes your application supports: - By declaring which screen sizes your application supports, you can ensure that only devices with the screens you support can download your application. Declaring support for different screen sizes can also affect how the system draws your application on larger screens—specifically, whether your application runs in screen compatibility mode.

To declare the screen sizes your application supports, you should include the `supports-screens` element in your manifest file.

- Provide different layouts for different screen sizes: - By default, Android resizes your application layout to fit the current device screen. In most cases, this works fine. In other cases, your UI might not look as good and might need adjustments for different screen sizes. For example, on a larger screen, you might want to adjust the position and size of some elements to take advantage of the additional screen space, or on a smaller screen, you might need to adjust sizes so that everything can fit on the screen.

4.1. INTRODUCTION TO MULTIPLE SCREEN SIZE AND ORIENTATION INTERFACES

The configuration qualifiers you can use to provide size-specific resources are `small`, `normal`, `large`, and `xlarge`. For example, layouts for an extra large screen should go in `layout-xlarge/`.

Beginning with Android 3.2 (API level 13), the above size groups are deprecated and you should instead use the `sw N dp` configuration qualifier to define the smallest available width required by your layout resources. For example, if your multi-pane tablet layout requires at least 600dp of screen width, you should place it in `layout-sw600dp/`. Using the new techniques for declaring layout resources is discussed further in the section about Declaring Tablet Layouts for Android 3.2.

- Provide different bitmap drawables for different screen densities: - By default, Android scales your bitmap drawables (`.png`, `.jpg`, and `.gif` files) and Nine-Patch drawables (`.9.png` files) so that they render at the appropriate physical size on each device. For example, if your application provides bitmap drawables only for the baseline, medium screen density (`mdpi`), then the system scales them up when on a high-density screen, and scales them down when on a low-density screen. This scaling can cause artifacts in the bitmaps. To ensure your bitmaps look their best, you should include alternative versions at different resolutions for different screen densities.

The configuration qualifiers you can use for density-specific resources are `ldpi` (low), `mdpi` (medium), `hdpi` (high), and `xhdpi` (extra high). For example, bitmaps for high-density screens should go in `drawable-hdpi/`.

4.1. INTRODUCTION TO MULTIPLE SCREEN SIZE AND ORIENTATION INTERFACES

Density qualifier	Description
ldpi	Resources for low-density (ldpi) screens (120dpi).
mdpi	Resources for medium-density (mdpi) screens (160dpi). (This is the baseline density.)
hdpi	Resources for high-density (hdpi) screens (240dpi).
xhdpi	Resources for extra-high-density (xhdpi) screens (320dpi).
xxhdpi	Resources for extra-extra-high-density (xxhdpi) screens (480dpi).
xxxhdpi	Resources for extra-extra-extra-high-density (xxxhdpi) uses (640dpi).
nodpi	Resources for all densities. These are density-independent resources. The system does not scale resources tagged with this qualifier, regardless of the current screen's density.
tvdpi	Resources for screens somewhere between mdpi and hdpi; approximately 213dpi. This is not considered a "primary" density group. It is mostly intended for televisions and most apps shouldn't need it—providing mdpi and hdpi resources is sufficient for most apps and the system will scale them as appropriate. If you find it necessary to provide tvdpi resources, you should size them at a factor of 1.33*mdpi. For example, a 100px x 100px image for mdpi screens should be 133px x 133px for tvdpi.

We can also use following methods to support multiple screen sizes and orientation:

1. Use "wrap_content" and "match_parent"
2. Use Relative Layout
3. Use Size Qualifiers
4. Use the Smallest-width Qualifier
5. Use Layout Aliases
6. Use Orientation Qualifiers
7. Use Nine-patch Bitmaps

4.2 User Interface Classes

All user interface elements in an Android app are built using View and ViewGroup objects. A View is an object that draws something on the screen that the user can interact with. A ViewGroup is an object that holds other View (and ViewGroup) objects in order to define the layout of the interface.

Android provides a collection of both View and ViewGroup subclasses that offer you common input controls (such as buttons and text fields) and various layout models (such as a linear or relative layout).

4.3 Android XML Layouts, Resource and Styles

4.3.1 Android XML

XML stands for Extensible Mark-up Language. XML is a very popular format and commonly used for sharing data on the internet. Android provides three types of XML parsers which are DOM, SAX and XMLPullParser. Among all of them android recommend XMLPullParser because it is efficient and easy to use. So we are going to use XMLPullParser for parsing XML. The first step is to identify the fields in the XML data in which you are interested in. For example. In the XML given below we interested in getting temperature only.

4.3.2 Layouts

A layout defines the structure for a user interface in your app, such as in an activity. All elements in the layout are built using a hierarchy of View and ViewGroup objects. A View usually draws something the user can see and interact with. Whereas a ViewGroup is an invisible container that defines the layout structure for View and other ViewGroup objects, as shown in figure 1.

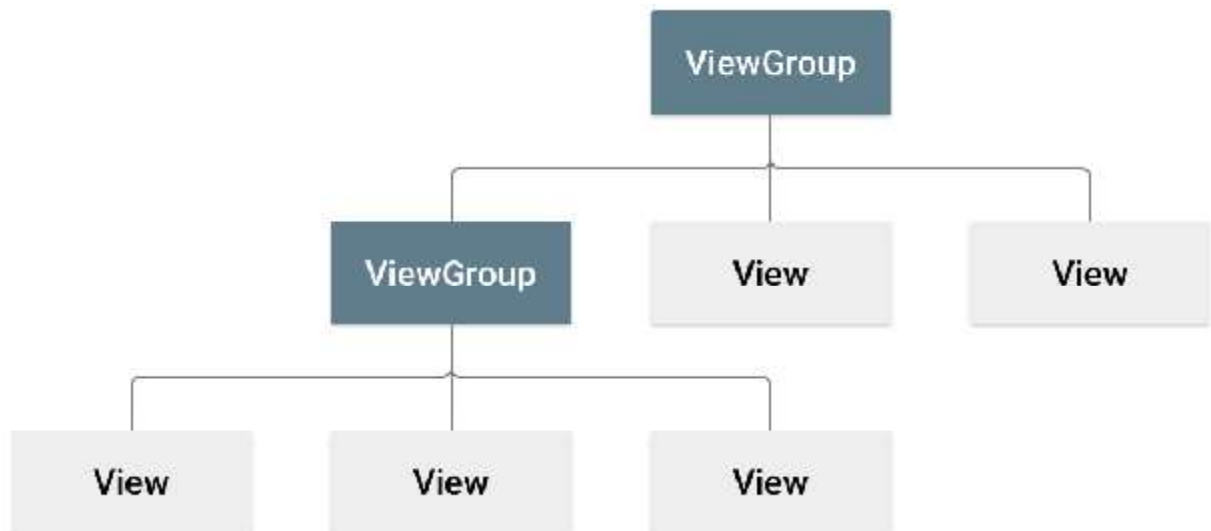


Figure 1. Illustration of a view hierarchy, which defines a UI layout

The View objects are usually called "widgets" and can be one of many subclasses, such as Button or TextView. The ViewGroup objects are usually called "layouts" can be one of many types that provide a different layout structure, such as LinearLayout or ConstraintLayout .

You can declare a layout in two ways:

- Declare UI elements in XML. Android provides a straightforward XML vocabulary that corresponds to the View classes and subclasses, such as those for widgets and layouts. You can also use Android Studio's Layout Editor to build your XML layout using a drag-and-drop interface.
- Instantiate layout elements at runtime. Your app can create View and ViewGroup objects (and manipulate their properties) programmatically.

Declaring your UI in XML allows you to separate the presentation of your app from the code that controls its behavior. Using XML files also makes it easy to provide different layouts for different screen sizes and orientations (discussed further in Supporting Different Screen Sizes).

The Android framework gives you the flexibility to use either or both of these methods to build your app's UI. For example, you can declare your app's default layouts in XML, and then modify the layout at runtime.

Different Types of Layout

Each subclass of the ViewGroup class provides a unique way to display the

4.3. ANDROID XML LAYOUTS, RESOURCE AND STYLES

views you nest within it. Below are some of the more common layout types that are built into the Android platform.

1. Linear Layout: - A layout that organizes its children into a single horizontal or vertical row. It creates a scrollbar if the length of the window exceeds the length of the screen.



2. Relative Layout: - Enables you to specify the location of child objects relative to each other (child A to the left of child B) or to the parent (aligned to the top of the parent).



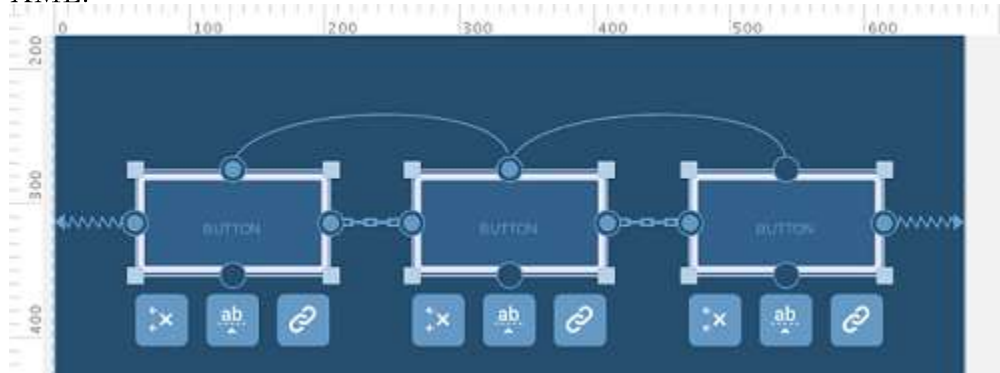
3. Web View: - Displays web pages.



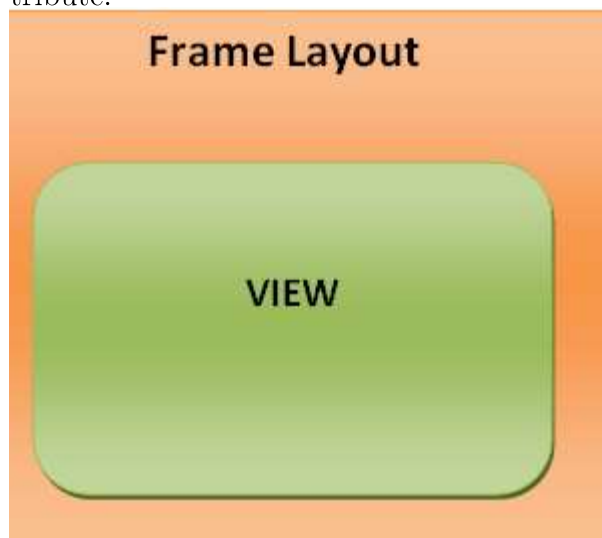
4. Constraint Layout: - ConstraintLayout allows you to create large and complex layouts with a flat view hierarchy (no nested view groups). It's similar to RelativeLayout in that all views are laid out according to relationships between sibling views and the parent layout, but it's more flexible than RelativeLayout and easier to use with Android Studio's Layout Editor.

4.3. ANDROID XML LAYOUTS, RESOURCE AND STYLES

All the power of `ConstraintLayout` is available directly from the Layout Editor's visual tools, because the layout API and the Layout Editor were specially built for each other. So you can build your layout with `ConstraintLayout` entirely by drag-and-dropping instead of editing the XML.



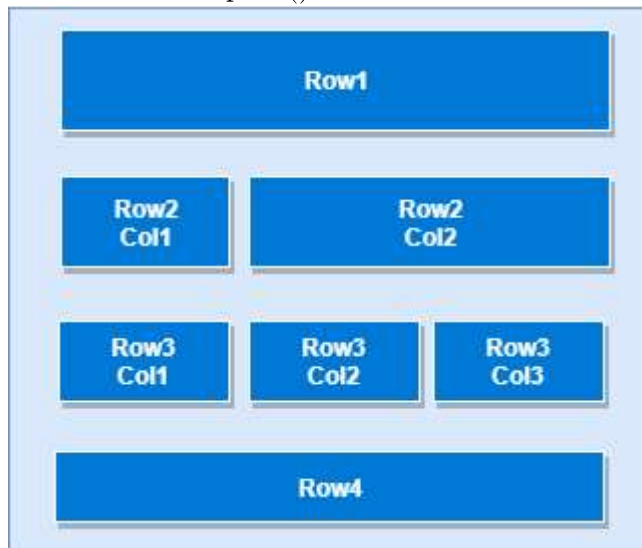
5. Frame Layout: - `FrameLayout` is designed to block out an area on the screen to display a single item. Generally, `FrameLayout` should be used to hold a single child view, because it can be difficult to organize child views in a way that's scalable to different screen sizes without the children overlapping each other. You can, however, add multiple children to a `FrameLayout` and control their position within the `FrameLayout` by assigning gravity to each child, using the `android:layout_gravity` attribute.



6. Table Layout: - A layout that arranges its children into rows and columns. A `TableLayout` consists of a number of `TableRow` objects, each defining a row (actually, you can have other children, which will

be explained below). `TableLayout` containers do not display border lines for their rows, columns, or cells. Each row has zero or more cells; each cell can hold one `View` object. The table has as many columns as the row with the most cells. A table can leave cells empty. Cells can span columns, as they can in HTML.

The width of a column is defined by the row with the widest cell in that column. However, a `TableLayout` can specify certain columns as shrinkable or stretchable by calling `setColumnShrinkable()` or `setColumnStretchable()`. If marked as shrinkable, the column width can be shrunk to fit the table into its parent object. If marked as stretchable, it can expand in width to fit any extra space. The total width of the table is defined by its parent container. It is important to remember that a column can be both shrinkable and stretchable. In such a situation, the column will change its size to always use up the available space, but never more. Finally, you can hide a column by calling `setColumnCollapsed()`.

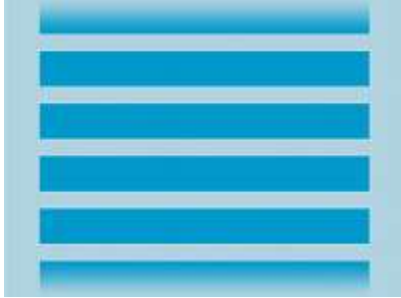


Building Layouts with an Adapter

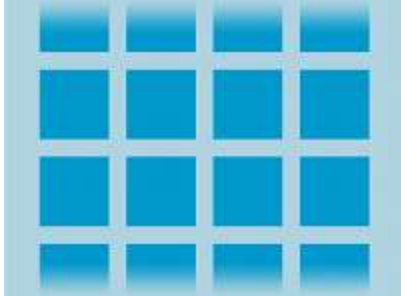
When the content for your layout is dynamic or not pre-determined, you can use a layout that subclasses `AdapterView` to populate the layout with views at runtime. A subclass of the `AdapterView` class uses an `Adapter` to bind data to its layout. The `Adapter` behaves as a middleman between the data source and the `AdapterView` layout—the `Adapter` retrieves the data (from a source such as an array or a database query) and converts each entry into a view that can be added into the `AdapterView` layout.

4.3. ANDROID XML LAYOUTS, RESOURCE AND STYLES

1. List View: - Displays a scrolling single column list.



2. Grid View: - Displays a scrolling grid of columns and rows.



UI Controls

We have a different type of UI controls available in android to implement user interface for our android applications

Following are the commonly used UI or input controls in android applications.

1. TextView
2. EditText
3. AutoCompleteTextView
4. Button
5. ImageButton
6. ToggleButton
7. CheckBox
8. RadioButton
9. RadioGroup
10. ProgressBar

4.3. ANDROID XML LAYOUTS, RESOURCE AND STYLES

11. Spinner
12. TimePicker
13. DatePicker
14. SeekBar
15. AlertDialog
16. Switch
17. RatingBar

Use all these UI controls and Layout with properties and methods in your Lab Work.

[Note: A practical example will be asked from UI Controls and Layouts]

4.3.3 Resources

Resources are the additional files and static content that your code uses, such as bitmaps, layout definitions, user interface strings and animation instructions. The Android resource system keeps track of all non-code assets associated with an application.

For any type of resource, you can specify default and multiple alternative resources for your application:

- Default resources are those that should be used regardless of the device configuration or when there are no alternative resources that match the current configuration.
- Alternative resources are those that you've designed for use with a specific configuration. To specify that a group of resources are for a specific configuration, append an appropriate configuration qualifier to the directory name.

The `res/` directory contains all the resources in various sub directories. Here we have an image resource, two layout resources, and a string resource file. Following table gives a detail about the resource directories supported inside project `res/` directory.

4.3. ANDROID XML LAYOUTS, RESOURCE AND STYLES

Directory	Resource Type
animator/	XML files that define property animations.
anim/	XML files that define tween animations. (Property animations can also be saved in this directory, but the animator/ directory is preferred for property animations to distinguish between the two types.)
color/	XML files that define a state list of colors. See Color State List Resource
drawable/	Bitmap files (.png, .9.png, .jpg, .gif) or XML files that are compiled into the following drawable resource subtypes: • Bitmap files • Nine-Patches (re-sizable bitmaps) • State lists • Shapes • Animation drawables • Other drawables
mipmap/	Drawable files for different launcher icon densities. For more information on managing launcher icons with mipmap/ folders, see Managing Projects Overview.
layout/	XML files that define a user interface layout. See Layout Resource.
menu/	XML files that define app menus, such as an Options Menu, Context Menu, or Sub Menu.
raw/	Arbitrary files to save in their raw form. To open these resources with a raw InputStream, call Resources.openRawResource() with the resource ID, which is R.raw.filename. However, if you need access to original file names and file hierarchy, you might consider saving some resources in the assets/ directory (instead of res/raw/). Files in assets/ aren't given a resource ID, so you can read them only using AssetManager.

values/	XML files that contain simple values, such as strings, integers, and colors. Whereas XML resource files in other res/ subdirectories define a single resource based on the XML filename, files in the values/ directory describe multiple resources. For a file in this directory, each child of the <code><resources></code> element defines a single resource. For example, a <code><string></code> element creates an <code>R.string</code> resource and a <code><color></code> element creates an <code>R.color</code> resource. Because each resource is defined with its own XML element, you can name the file whatever you want and place different resource types in one file. However, for clarity, you might want to place unique resource types in different files. For example, here are some filename conventions for resources you can create in this directory: • arrays.xml for resource arrays (typed arrays). • colors.xml for color values • dimens.xml for dimension values. • strings.xml for string values. • styles.xml for styles.
xml/	Arbitrary XML files that can be read at runtime by calling <code>Resources.getXML()</code>. Various XML configuration files must be saved here, such as a searchable configuration.
font/	Font files with extensions such as .ttf, .otf, or .ttc, or XML files that include a <code><font-family></code> element. For more information about fonts as resources, go to Fonts in XML.

4.3.4 Styles

A style is a collection of properties that specify the look and format for a View or window. A style can specify properties such as height, padding, font color, font size, background color, and much more. A style is defined in an XML resource that is separate from the XML that specifies the layout.

Styles in Android share a similar philosophy to cascading stylesheets in web design they allow you to separate the design from the content. For example, by using a style, you can take this layout XML:

A theme is a style applied to an entire Activity or application, rather than an individual View (as in the example above). When a style is applied

4.4. ANDROID 3RD PARTY UI/UX LIBRARIES

as a theme, every View in the Activity or application will apply each style property that it supports. To create a set of styles, save an XML file in the `res/values/` directory of android project. The name of the XML file is arbitrary, but it must use the `.xml` extension and be saved in the `res/values/` folder. The root node of the XML file must be `resources`. For each style we want to create, add a `style` element to the file with a name that uniquely identifies the style (this attribute is required). Then add an `item` element for each property of that style, with a name that declares the style property and a value to go with it (this attribute is required). The value for the `item` can be a keyword string, a hex color, a reference to another resource type, or other value depending on the style property. Here's an example file with a single style:

4.4 Android 3rd Party UI/UX Libraries

Google wants Android to be in everything that has web connectivity and to that effect, the company has unveiled what third party apps might one day look like inside your car. Android for the car will be called Android Auto and the image you see here is what the apps might one day look like in the car, assuming your car supports Android Auto. Google did the same thing with Android Wear where the apps for Wear aren't complete apps, rather they are used to control what content from an app on your smartphone you can see on the wearable device. There are many third-party libraries for Android but several of them are "must have" libraries that are extremely popular and are often used in almost any Android project. Each has different purposes but all of them make life as a developer much more pleasant.